

Table of Contents

1. Rmq
2. K Th
3. Mex
4. Games_on_graphs
5. Sprague Grundy Nim
6. Schedule_one_machine
7. Schedule_two_machines
8. Schedule With Completion Duration
9. Maximum_average_segment
10. Tortoise_and_hare
11. Josephus_problem
12. 15 Puzzle
13. Stern_brocot_tree_farey_sequences

Translated

From: e-maxx.ru

Range Minimum Query

You are given an array $A[1..N]$. You have to answer incoming queries of the form (L, R) , which ask to find the minimum element in array A between positions L and R inclusive.

RMQ can appear in problems directly or can be applied in some other tasks, e.g. the [Lowest Common Ancestor](#) problem.

Solution

There are lots of possible approaches and data structures that you can use to solve the RMQ task.

The ones that are explained on this site are listed below.

First the approaches that allow modifications to the array between answering queries.

- [Sqrt-decomposition](#) - answers each query in $O(\sqrt{N})$, preprocessing done in $O(N)$. Pros: a very simple data structure. Cons: worse complexity.
- [Segment tree](#) - answers each query in $O(\log N)$, preprocessing done in $O(N)$. Pros: good time complexity. Cons: larger amount of code compared to the other data structures.
- [Fenwick tree](#) - answers each query in $O(\log N)$, preprocessing done in $O(N \log N)$. Pros: the shortest code, good time complexity. Cons: Fenwick tree can only be used for queries with $L = 1$, so it is not applicable to many problems.

And here are the approaches that only work on static arrays, i.e. it is not possible to change a value in the array without recomputing the complete data structure.

- [Sparse Table](#) - answers each query in $O(1)$, preprocessing done in $O(N \log N)$.
Pros: simple data structure, excellent time complexity.
- [Sqrt Tree](#) - answers queries in $O(1)$, preprocessing done in $O(N \log \log N)$. Pros: fast. Cons: Complicated to implement.
- [Disjoint Set Union / Arpa's Trick](#) - answers queries in $O(1)$, preprocessing in $O(n)$.
Pros: short, fast. Cons: only works if all queries are known in advance, i.e. only supports off-line processing of the queries.
- [Cartesian Tree](#) and [Farach-Colton and Bender algorithm](#) - answers queries in $O(1)$, preprocessing in $O(n)$. Pros: optimal complexity. Cons: large amount of code.

Note: Preprocessing is the preliminary processing of the given array by building the corresponding data structure for it.

Practice Problems

- [SPOJ: Range Minimum Query](#)
- [CODECHEF: Chef And Array](#)
- [Codeforces: Array Partition](#)

Contributors:

[Jakob Kogler](#) (47.62%) [Anthony](#) (19.05%) [Oleksandr Kulkov](#) (14.29%)
[Mariia Mykhailova](#) (9.52%) [Kartikay singh](#) (7.14%) [Tanmay Sinha](#) (2.38%)

Translated

From: e-maxx.ru

K th order statistic in $O(N)$

Given an array A of size N and a number K . The problem is to find K -th largest number in the array, i.e., K -th order statistic.

The basic idea - to use the idea of quick sort algorithm. Actually, the algorithm is simple, it is more difficult to prove that it runs in an average of $O(N)$, in contrast to the quick sort.

Implementation (not recursive)

```
template <class T>
T order_statistics (std::vector<T> a, unsigned n, unsigned k)
{
    using std::swap;
    for (unsigned l=1, r=n; ; )
    {
        if (r <= l+1)
        {
            // the current part size is either 1 or 2, so it is easy to
            find the answer
            if (r == l+1 && a[r] < a[l])
                swap (a[l], a[r]);
            return a[k];
        }

        // ordering a[l], a[l+1], a[r]
        unsigned mid = (l + r) >> 1;
        swap (a[mid], a[l+1]);
        if (a[l] > a[r])
            swap (a[l], a[r]);
        if (a[l+1] > a[r])
            swap (a[l+1], a[r]);
        if (a[l] > a[l+1])
            swap (a[l], a[l+1]);

        // performing division
```

```

// barrier is a[l + 1], i.e. median among a[l], a[l + 1], a[r]
unsigned
    i = l+1,
    j = r;
const T
    cur = a[l+1];
for (;;)
{
    while (a[++i] < cur) ;
    while (a[--j] > cur) ;
    if (i > j)
        break;
    swap (a[i], a[j]);
}

// inserting the barrier
a[l+1] = a[j];
a[j] = cur;

// we continue to work in that part, which must contain the
required element
if (j >= k)
    r = j-1;
if (j <= k)
    l = i;
}
}

```

Notes

- The randomized algorithm above is named [quickselect](#). You should do random shuffle on A before calling it or use a random element as a barrier for it to run properly. There are also deterministic algorithms that solve the specified problem in linear time, such as [median of medians](#).
- [std::nth_element](#) solves this in C++ but gcc's implementation runs in worst case $O(n \log n)$ time.
- Finding K smallest elements can be reduced to finding K -th element with a linear overhead, as they're exactly the elements that are smaller than K -th.

Practice Problems

- [Leetcode: Kth Largest Element in an Array](#)
- [CODECHEF: Median](#)

Contributors:

[Yerzhan Dyussenaliyev](#) (73.68%) [Oleksandr Kulkov](#) (11.84%) [Jakob Kogler](#) (5.26%)
[Bhuvnesh Jain](#) (2.63%) [shark_s](#) (2.63%) [Michael Hayter](#) (1.32%) [Shadab Zafar](#) (1.32%)
[random1001guy](#) (1.32%)

Original

MEX (minimal excluded) of a sequence

Given an array A of size N . You have to find the minimal non-negative element that is not present in the array. That number is commonly called the **MEX** (minimal excluded).

$$\text{mex}(\{0, 1, 2, 4, 5\}) = 3$$

$$\text{mex}(\{0, 1, 2, 3, 4\}) = 5$$

$$\text{mex}(\{1, 2, 3, 4, 5\}) = 0$$

Notice, that the MEX of an array of size N can never be bigger than N itself.

The easiest approach is to create a set of all elements in the array A , so that we can quickly check if a number is part of the array or not. Then we can check all numbers from 0 to N , if the current number is not present in the set, return it.

Implementation

The following algorithm runs in $O(N \log N)$ time.

```
int mex(vector<int> const& A) {
    set<int> b(A.begin(), A.end());

    int result = 0;
    while (b.count(result))
        ++result;
    return result;
}
```

If an algorithm requires a $O(N)$ MEX computation, it is possible by using a boolean vector instead of a set. Notice, that the array needs to be as big as the biggest possible array size.

```

int mex(vector<int> const& A) {
    static bool used[MAX_N+1] = { 0 };

    // mark the given numbers
    for (int x : A) {
        if (x <= MAX_N)
            used[x] = true;
    }

    // find the mex
    int result = 0;
    while (used[result])
        ++result;

    // clear the array again
    for (int x : A) {
        if (x <= MAX_N)
            used[x] = false;
    }

    return result;
}

```

This approach is fast, but only works well if you have to compute the MEX once. If you need to compute the MEX over and over, e.g. because your array keeps changing, then it is not effective. For that, we need something better.

MEX with array updates

In the problem you need to change individual numbers in the array, and compute the new MEX of the array after each such update.

There is a need for a better data structure that handles such queries efficiently.

One approach would be take the frequency of each number from 0 to N , and build a tree-like data structure over it. E.g. a segment tree or a treap. Each node represents a range of numbers, and together to total frequency in the range, you additionally store the amount of distinct numbers in that range. It's possible to update this data structure in $O(\log N)$ time, and also find the MEX in $O(\log N)$ time, by doing a binary search for the MEX. If the node representing the range $[0, \lfloor N/2 \rfloor)$ doesn't contain $\lfloor N/2 \rfloor$ many distinct numbers, then one is missing and the MEX is smaller than $\lfloor N/2 \rfloor$, and you can

recurse in the left branch of the tree. Otherwise it is at least $\lfloor N/2 \rfloor$, and you can recurse in the right branch of the tree.

It's also possible to use the standard library data structures `map` and `set` (based on an approach explained [here](#)). With a `map` we will remember the frequency of each number, and with the `set` we represent the numbers that are currently missing from the array. Since a `set` is ordered, `*set.begin()` will be the MEX. In total we need $O(N \log N)$ precomputation, and afterwards the MEX can be computed in $O(1)$ and an update can be performed in $O(\log N)$.

```
class Mex {
private:
    map<int, int> frequency;
    set<int> missing_numbers;
    vector<int> A;

public:
    Mex(vector<int> const& A) : A(A) {
        for (int i = 0; i <= A.size(); i++)
            missing_numbers.insert(i);

        for (int x : A) {
            ++frequency[x];
            missing_numbers.erase(x);
        }
    }

    int mex() {
        return *missing_numbers.begin();
    }

    void update(int idx, int new_value) {
        if (--frequency[A[idx]] == 0)
            missing_numbers.insert(A[idx]);
        A[idx] = new_value;
        ++frequency[new_value];
        missing_numbers.erase(new_value);
    }
};
```

Practice Problems

- [AtCoder: Neq Min](#)

- [Codeforces: Informatics in MAC](#)
- [Codeforces: Replace by MEX](#)
- [Codeforces: Vitya and Strange Lesson](#)
- [Codeforces: MEX Queries](#)

Contributors:

[Jakob Kogler](#) (78.57%) [Alex Maksimov](#) (19.05%) [Oleksandr Kulkov](#) (1.59%)
[Harsh Kumar](#) (0.79%)

Translated

From: e-maxx.ru

Games on arbitrary graphs

Let a game be played by two players on an arbitrary graph G . I.e. the current state of the game is a certain vertex. The players perform moves by turns, and move from the current vertex to an adjacent vertex using a connecting edge. Depending on the game, the person that is unable to move will either lose or win the game.

We consider the most general case, the case of an arbitrary directed graph with cycles. It is our task to determine, given an initial state, who will win the game if both players play with optimal strategies or determine that the result of the game will be a draw.

We will solve this problem very efficiently. We will find the solution for all possible starting vertices of the graph in linear time with respect to the number of edges: $O(m)$.

Description of the algorithm

We will call a vertex a winning vertex, if the player starting at this state will win the game, if they play optimally (regardless of what turns the other player makes). Similarly, we will call a vertex a losing vertex, if the player starting at this vertex will lose the game, if the opponent plays optimally.

For some of the vertices of the graph, we already know in advance that they are winning or losing vertices: namely all vertices that have no outgoing edges.

Also we have the following **rules**:

- if a vertex has an outgoing edge that leads to a losing vertex, then the vertex itself is a winning vertex.
- if all outgoing edges of a certain vertex lead to winning vertices, then the vertex itself is a losing vertex.

- if at some point there are still undefined vertices, and neither will fit the first or the second rule, then each of these vertices, when used as a starting vertex, will lead to a draw if both player play optimally.

Thus, we can define an algorithm which runs in $O(nm)$ time immediately. We go through all vertices and try to apply the first or second rule, and repeat.

However, we can accelerate this procedure, and get the complexity down to $O(m)$.

We will go over all the vertices, for which we initially know if they are winning or losing states. For each of them, we start a [depth first search](#). This DFS will move back over the reversed edges. First of all, it will not enter vertices which already are defined as winning or losing vertices. And further, if the search goes from a losing vertex to an undefined vertex, then we mark this one as a winning vertex, and continue the DFS using this new vertex. If we go from a winning vertex to an undefined vertex, then we must check whether all edges from this one leads to winning vertices. We can perform this test in $O(1)$ by storing the number of edges that lead to a winning vertex for each vertex. So if we go from a winning vertex to an undefined one, then we increase the counter, and check if this number is equal to the number of outgoing edges. If this is the case, we can mark this vertex as a losing vertex, and continue the DFS from this vertex. Otherwise we don't know yet, if this vertex is a winning or losing vertex, and therefore it doesn't make sense to keep continuing the DFS using it.

In total we visit every winning and every losing vertex exactly once (undefined vertices are not visited), and we go over each edge also at most one time. Hence the complexity is $O(m)$.

Implementation

Here is the implementation of such a DFS. We assume that the variable `adj_rev` stores the adjacency list for the graph in **reversed** form, i.e. instead of storing the edge (i, j) of the graph, we store (j, i) . Also for each vertex we assume that the outgoing degree is already computed.

```
vector<vector<int>> adj_rev;

vector<bool> winning;
```

```

vector<bool> losing;
vector<bool> visited;
vector<int> degree;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}

```

Example: "Policeman and thief"

Here is a concrete example of such a game.

There is $m \times n$ board. Some of the cells cannot be entered. The initial coordinates of the police officer and of the thief are known. One of the cells is the exit. If the policeman and the thief are located at the same cell at any moment, the policeman wins. If the thief is at the exit cell (without the policeman also being on the cell), then the thief wins. The policeman can walk in all 8 directions, the thief only in 4 (along the coordinate axis). Both the policeman and the thief will take turns moving. However they also can skip a turn if they want to. The first move is made by the policeman.

We will now **construct the graph**. For this we must formalize the rules of the game. The current state of the game is determined by the coordinates of the police offices P , the coordinates of the thief T , and also by whose turn it is, let's call this variable P_{turn} (which is true when it is the policeman's turn). Therefore a vertex of the graph is determined by the triple (P, T, P_{turn}) . The graph then can be easily constructed, simply by following the rules of the game.

Next we need to determine which vertices are winning and which are losing ones initially. There is a **subtle point** here. The winning / losing vertices depend, in addition to

the coordinates, also on P_{turn} - whose turn it. If it is the policeman's turn, then the vertex is a winning vertex, if the coordinates of the policeman and the thief coincide, and the vertex is a losing one if it is not a winning one and the thief is on the exit vertex. If it is the thief's turn, then a vertex is a losing vertex, if the coordinates of the two players coincide, and it is a winning vertex if it is not a losing one, and the thief is at the exit vertex.

The only point before implementing is not, that you need to decide if you want to build the graph **explicitly** or just construct it **on the fly**. On one hand, building the graph explicitly will be a lot easier and there is less chance of making mistakes. On the other hand, it will increase the amount of code and the running time will be slower than if you build the graph on the fly.

The following implementation will construct the graph explicitly:

```
struct State {
    int P, T;
    bool Pstep;
};

vector<State> adj_rev[100][100][2]; // [P][T][Pstep]
bool winning[100][100][2];
bool losing[100][100][2];
bool visited[100][100][2];
int degree[100][100][2];

void dfs(State v) {
    visited[v.P][v.T][v.Pstep] = true;
    for (State u : adj_rev[v.P][v.T][v.Pstep]) {
        if (!visited[u.P][u.T][u.Pstep]) {
            if (losing[v.P][v.T][v.Pstep])
                winning[u.P][u.T][u.Pstep] = true;
            else if (--degree[u.P][u.T][u.Pstep] == 0)
                losing[u.P][u.T][u.Pstep] = true;
            else
                continue;
            dfs(u);
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
```

```

vector<string> a(n);
for (int i = 0; i < n; i++)
    cin >> a[i];

for (int P = 0; P < n*m; P++) {
    for (int T = 0; T < n*m; T++) {
        for (int Pstep = 0; Pstep <= 1; Pstep++) {
            int Px = P/m, Py = P%m, Tx = T/m, Ty = T%m;
            if (a[Px][Py]=='*' || a[Tx][Ty]=='*')
                continue;

            bool& win = winning[P][T][Pstep];
            bool& lose = losing[P][T][Pstep];
            if (Pstep) {
                win = Px==Tx && Py==Ty;
                lose = !win && a[Tx][Ty] == 'E';
            } else {
                lose = Px==Tx && Py==Ty;
                win = !lose && a[Tx][Ty] == 'E';
            }
            if (win || lose)
                continue;

            State st = {P,T,!Pstep};
            adj_rev[P][T][Pstep].push_back(st);
            st.Pstep = Pstep;
            degree[P][T][Pstep]++;

            const int dx[] = {-1, 0, 1, 0, -1, -1, 1, 1};
            const int dy[] = {0, 1, 0, -1, -1, 1, -1, 1};
            for (int d = 0; d < (Pstep ? 8 : 4); d++) {
                int PPx = Px, PPy = Py, TTx = Tx, TTy = Ty;
                if (Pstep) {
                    PPx += dx[d];
                    PPy += dy[d];
                } else {
                    TTx += dx[d];
                    TTy += dy[d];
                }

                if (PPx >= 0 && PPx < n && PPy >= 0 && PPy < m &&
                    a[PPx][PPy] != '*' &&
                    TTx >= 0 && TTx < n && TTy >= 0 && TTy < m &&
                    a[TTx][TTy] != '*')
                {
                    adj_rev[PPx*m+PPy][TTx*m+TTy]
[!Pstep].push_back(st);
                    ++degree[P][T][Pstep];
                }
            }
        }
    }
}

```

```

    }
    }
}

for (int P = 0; P < n*m; P++) {
    for (int T = 0; T < n*m; T++) {
        for (int Pstep = 0; Pstep <= 1; Pstep++) {
            if ((winning[P][T][Pstep] || losing[P][T][Pstep]) &&
!visited[P][T][Pstep])
                dfs({P, T, (bool)Pstep});
        }
    }
}

int P_st, T_st;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (a[i][j] == 'P')
            P_st = i*m+j;
        else if (a[i][j] == 'T')
            T_st = i*m+j;
    }
}

if (winning[P_st][T_st][true]) {
    cout << "Police catches the thief" << endl;
} else if (losing[P_st][T_st][true]) {
    cout << "The thief escapes" << endl;
} else {
    cout << "Draw" << endl;
}
}

```

Contributors:

[Jakob Kogler](#) (94.17%)
[Abhijeet Krishnan](#) (3.14%)
[Oleksandr Kulkov](#) (2.69%)

Translated

From: e-maxx.ru

Sprague-Grundy theorem. Nim

Introduction

This theorem describes the so-called **impartial** two-player game, i.e. those in which the available moves and winning/losing depends only on the state of the game. In other words, the only difference between the two players is that one of them moves first.

Additionally, we assume that the game has **perfect information**, i.e. no information is hidden from the players (they know the rules and the possible moves).

It is assumed that the game is **finite**, i.e. after a certain number of moves, one of the players will end up in a losing position — from which they can't move to another position. On the other side, the player who set up this position for the opponent wins. Understandably, there are no draws in this game.

Such games can be completely described by a *directed acyclic graph*: the vertices are game states and the edges are transitions (moves). A vertex without outgoing edges is a losing vertex (a player who must make a move from this vertex loses).

Since there are no draws, we can classify all game states as either **winning** or **losing**. Winning states are those from which there is a move that causes inevitable defeat of the other player, even with their best response. Losing states are those from which all moves lead to winning states for the other player. Summarizing, a state is winning if there is at least one transition to a losing state and is losing if there isn't at least one transition to a losing state.

Our task is to classify the states of a given game.

The theory of such games was independently developed by Roland Sprague in 1935 and Patrick Michael Grundy in 1939.

Nim

This game obeys the restrictions described above. Moreover, *any* perfect-information impartial two-player game can be reduced to the game of Nim. Studying this game will allow us to solve all other similar games, but more on that later.

Historically this game was popular in ancient times. Its origin is probably in China — or at least the game *Jianshizi* is very similar to it. In Europe the earliest references to it are from the 16th century. The name was given by Charles Bouton, who in 1901 published a full analysis of this game.

Game description

There are several piles, each with several stones. In a move a player can take any positive number of stones from any one pile and throw them away. A player loses if they can't make a move, which happens when all the piles are empty.

The game state is unambiguously described by a multiset of positive integers. A move consists of strictly decreasing a chosen integer (if it becomes zero, it is removed from the set).

The solution

The solution by Charles L. Bouton looks like this:

Theorem. The current player has a winning strategy if and only if the xor-sum of the pile sizes is non-zero. The xor-sum of a sequence a is $a_1 \oplus a_2 \oplus \dots \oplus a_n$, where $\oplus$ is the *bitwise exclusive or*.

Proof. The key to the proof is the presence of a **symmetric strategy for the opponent**. We show that a once in a position with the xor-sum equal to zero, the player won't be able to make it non-zero in the long term — if they transition to a position with a non-zero xor-sum, the opponent will always have a move returning the xor-sum back to zero.

We will prove the theorem by mathematical induction.

For an empty Nim (where all the piles are empty i.e. the multiset is empty) the xor-sum is zero and the theorem is true.

Now suppose we are in a non-empty state. Using the assumption of induction (and the acyclicity of the game) we assume that the theorem is proven for all states reachable from the current one.

Then the proof splits into two parts: if for the current position the xor-sum $s = 0$, we have to prove that this state is losing, i.e. all reachable states have xor-sum $t \neq 0$. If $s \neq 0$, we have to prove that there is a move leading to a state with $t = 0$.

- Let $s = 0$ and let's consider any move. This move reduces the size of a pile x to a size y . Using elementary properties of $\oplus$, we have

$$t = s \oplus x \oplus y = 0 \oplus x \oplus y = x \oplus y$$

Since $y < x$, $y \oplus x$ can't be zero, so $t \neq 0$. That means any reachable state is a winning one (by the assumption of induction), so we are in a losing position.

- Let $s \neq 0$. Consider the binary representation of the number s . Let d be the index of its leading (biggest value) non-zero bit. Our move will be on a pile whose size's bit number d is set (it must exist, otherwise the bit wouldn't be set in s). We will reduce its size x to $y = x \oplus s$. All bits at positions greater than d in x and y match and bit d is set in x but not set in y . Therefore, $y < x$, which is all we need for a move to be legal. Now we have:

$$t = s \oplus x \oplus y = s \oplus x \oplus (s \oplus x) = 0$$

This means we found a reachable losing state (by the assumption of induction) and the current state is winning.

Corollary. Any state of Nim can be replaced by an equivalent state as long as the xor-sum doesn't change. Moreover, when analyzing a Nim with several piles, we can replace it with a single pile of size s .

Misère Game

In a **misère game**, the goal of the game is opposite, so the player who removes the last stick loses the game. It turns out that the misère nim game can be optimally played

almost like a standard nim game. The idea is to first play the misère game like the standard game, but change the strategy at the end of the game. The new strategy will be introduced in a situation where each heap would contain at most one stick after the next move. In the standard game, we should choose a move after which there is an even number of heaps with one stick. However, in the misère game, we choose a move so that there is an odd number of heaps with one stick. This strategy works because a state where the strategy changes always appears in the game, and this state is a winning state, because it contains exactly one heap that has more than one stick so the nim sum is not 0.

The equivalence of impartial games and Nim (Sprague-Grundy theorem)

Now we will learn how to find, for any game state of any impartial game, a corresponding state of Nim.

Lemma about Nim with increases

We consider the following modification to Nim: we also allow **adding stones to a chosen pile**. The exact rules about how and when increasing is allowed **do not interest us**, however the rules should keep our game **acyclic**. In later sections, example games are considered.

Lemma. The addition of increasing to Nim doesn't change how winning and losing states are determined. In other words, increases are useless, and we don't have to use them in a winning strategy.

Proof. Suppose a player added stones to a pile. Then his opponent can simply undo his move — decrease the number back to the previous value. Since the game is acyclic, sooner or later the current player won't be able to use an increase move and will have to do the usual Nim move.

Sprague-Grundy theorem

Let's consider a state v of a two-player impartial game and let v_i be the states reachable from it (where $i \in \{1, 2, \dots, k\}, k \geq 0$). To this state, we can assign a fully equivalent game of Nim with one pile of size x . The number x is called the Grundy value or nim-value of state v .

Moreover, this number can be found in the following recursive way:

$$x = \text{mex} \{x_1, \dots, x_k\},$$

where x_i is the Grundy value for state v_i and the function *mex* (*minimum excludant*) is the smallest non-negative integer not found in the given set.

Viewing the game as a graph, we can gradually calculate the Grundy values starting from vertices without outgoing edges. Grundy value being equal to zero means a state is losing.

Proof. We will use a proof by induction.

For vertices without a move, the value x is the *mex* of an empty set, which is zero. That is correct, since an empty Nim is losing.

Now consider any other vertex v . By induction, we assume the values x_i corresponding to its reachable vertices are already calculated.

Let $p = \text{mex} \{x_1, \dots, x_k\}$. Then we know that for any integer $i \in [0, p)$ there exists a reachable vertex with Grundy value i . This means v is **equivalent to a state of the game of Nim with increases with one pile of size p** . In such a game we have transitions to piles of every size smaller than p and possibly transitions to piles with sizes greater than p . Therefore, p is indeed the desired Grundy value for the currently considered state.

Application of the theorem

Finally, we describe an algorithm to determine the win/loss outcome of a game, which is applicable to any impartial two-player game.

To calculate the Grundy value of a given state you need to:

- Get all possible transitions from this state
- Each transition can lead to a **sum of independent games** (one game in the degenerate case). Calculate the Grundy value for each independent game and xor-sum them. Of course xor does nothing if there is just one game.
- After we calculated Grundy values for each transition we find the state's value as the mex of these numbers.
- If the value is zero, then the current state is losing, otherwise it is winning.

In comparison to the previous section, we take into account the fact that there can be transitions to combined games. We consider them a Nim with pile sizes equal to the independent games' Grundy values. We can xor-sum them just like usual Nim according to Bouton's theorem.

Patterns in Grundy values

Very often when solving specific tasks using Grundy values, it may be beneficial to **study the table of the values** in search of patterns.

In many games, which may seem rather difficult for theoretical analysis, the Grundy values turn out to be periodic or of an easily understandable form. In the overwhelming majority of cases the observed pattern turns out to be true and can be proved by induction if desired.

However, Grundy values are far from *always* containing such regularities and even for some very simple games, the problem asking if those regularities exist is still open (e.g. "Grundy's game").

Example games

Crosses-crosses

The rules. Consider a checkered strip of size $1 \times n$. In one move, the player must put one cross, but it is forbidden to put two crosses next to each other (in adjacent cells). As usual, the player without a valid move loses.

The solution. When a player puts a cross in any cell, we can think of the strip being split into two independent parts: to the left of the cross and to the right of it. In this case, the cell with a cross, as well as its left and right neighbours are destroyed – nothing more can be put in them. Therefore, if we number the cells from 1 to n then putting the cross in position $1 < i < n$ breaks the strip into two strips of length $i - 2$ and $n - i - 1$ i.e. we go to the sum of games $i - 2$ and $n - i - 1$. For the edge case of the cross being marked on position 1 or n , we go to the game $n - 2$.

Thus, the Grundy value $g(n)$ has the form:

$$g(n) = \text{mex}(\{g(n - 2)\} \cup \{g(i - 2) \oplus g(n - i - 1) \mid 2 \leq i \leq n - 1\}).$$

So we've got a $O(n^2)$ solution.

In fact, $g(n)$ has a period of length 34 starting with $n = 52$.

Practice Problems

- [KATTIS S-Nim](#)
- [CodeForces - Marbles \(2018-2019 ACM-ICPC Brazil Subregional\)](#)
- [KATTIS - Cuboid Slicing Game](#)
- [HackerRank - Tower Breakers, Revisited!](#)
- [HackerRank - Tower Breakers, Again!](#)
- [HackerRank - Chessboard Game, Again!](#)
- [Atcoder - ABC368F - Dividing Game](#)

Contributors:

[wiktorkuchta](#) (69.96%)
 [Abhijeet Krishnan](#) (13.9%)
 [Anikett](#) (4.93%)
 [iagorrr](#) (3.14%)
[Jakob Kogler](#) (2.69%)
 [Oleksandr Kulkov](#) (2.69%)
 [Bergur Snorrason](#) (1.35%)
[JustAnAverageGuy](#) (0.45%)
 [Tanner Hoke](#) (0.45%)
 [conlacda](#) (0.45%)

Translated

From: e-maxx.ru

Scheduling jobs on one machine

This task is about finding an optimal schedule for n jobs on a single machine, if the job i can be processed in t_i time, but for the t seconds waiting before processing the job a penalty of $f_i(t)$ has to be paid.

Thus the task asks to find such an permutation of the jobs, so that the total penalty is minimal. If we denote by π the permutation of the jobs (π_1 is the first processed item, π_2 the second, etc.), then the total penalty is equal to:

$$F(\pi) = f_{\pi_1}(0) + f_{\pi_2}(t_{\pi_1}) + f_{\pi_3}(t_{\pi_1} + t_{\pi_2}) + \dots + f_{\pi_n}\left(\sum_{i=1}^{n-1} t_{\pi_i}\right)$$

Solutions for special cases

Linear penalty functions

First we will solve the problem in the case that all penalty functions $f_i(t)$ are linear, i.e. they have the form $f_i(t) = c_i \cdot t$, where c_i is a non-negative number. Note that these functions don't have a constant term. Otherwise we can sum up all constant term, and resolve the problem without them.

Let us fixate some permutation π , and take an index $i = 1 \dots n - 1$. Let the permutation π' be equal to the permutation π with the elements i and $i + 1$ switched. Let's see how much the penalty changed.

$$F(\pi') - F(\pi) =$$

It is easy to see that the changes only occur in the i -th and $(i + 1)$ -th summands:

$$\begin{aligned}
 &= c_{\pi'_i} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi'_{i+1}} \cdot \sum_{k=1}^i t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^i t_{\pi_k} \\
 &= c_{\pi_{i+1}} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi_i} \cdot \sum_{k=1}^i t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^i t_{\pi_k} \\
 &= c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i}
 \end{aligned}$$

It is easy to see, that if the schedule π is optimal, than any change in it leads to an increased penalty (or to the identical penalty), therefore for the optimal schedule we can write down the following condition:

$$c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i} \geq 0 \quad \forall i = 1 \dots n - 1$$

And after rearranging we get:

$$\frac{c_{\pi_i}}{t_{\pi_i}} \geq \frac{c_{\pi_{i+1}}}{t_{\pi_{i+1}}} \quad \forall i = 1 \dots n - 1$$

Thus we obtain the **optimal schedule** by simply **sorting** the jobs by the fraction $\frac{c_i}{t_i}$ in non-ascending order.

It should be noted, that we constructed this algorithm by the so-called **permutation method**: we tried to swap two adjacent elements, calculated how much the penalty changed, and then derived the algorithm for finding the optimal method.

Exponential penalty function

Let the penalty function look like this:

$$f_i(t) = c_i \cdot e^{a \cdot t},$$

where all numbers c_i are non-negative and the constant α is positive.

By applying the permutation method, it is easy to determine that the jobs must be sorted in non-ascending order of the value:

$$v_i = \frac{1 - e^{\alpha \cdot t_i}}{c_i}$$

Identical monotone penalty function

In this case we consider the case that all $f_i(t)$ are equal, and this function is monotone increasing.

It is obvious that in this case the optimal permutation is to arrange the jobs by non-descending processing time t_i .

The Livshits-Kladov theorem

The Livshits-Kladov theorem establishes that the permutation method is only applicable for the above mentioned three cases, i.e.:

- Linear case: $f_i(t) = c_i(t) + d_i$, where c_i are non-negative constants,
- Exponential case: $f_i(t) = c_i \cdot e_{\alpha \cdot t} + d_i$, where c_i and α are positive constants,
- Identical case: $f_i(t) = \phi(t)$, where ϕ is a monotone increasing function.

In all other cases the method cannot be applied.

The theorem is proven under the assumption that the penalty functions are sufficiently smooth (the third derivatives exists).

In all three case we apply the permutation method, through which the desired optimal schedule can be found by sorting, hence in $O(n \log n)$ time.

Contributors:

[Jakob Kogler](#) (88.89%) [Oleksandr Kulkov](#) (7.41%) [vxw77](#) (2.47%) [P4R7H0 K2](#) (1.23%)

Translated

From: e-maxx.ru

Scheduling jobs on two machines

This task is about finding an optimal schedule for n jobs on two machines. Every item must first be processed on the first machine, and afterwards on the second one. The i -th job takes a_i time on the first machine, and b_i time on the second machine. Each machine can only process one job at a time.

We want to find the optimal order of the jobs, so that the final processing time is the minimum possible.

This solution that is discussed here is called Johnson's rule (named after S. M. Johnson).

It is worth noting, that the task becomes NP-complete, if we have more than two machines.

Construction of the algorithm

Note first, that we can assume that the order of jobs for the first and the second machine have to coincide. In fact, since the jobs for the second machine become available after processing them at the first, and if there are several jobs available for the second machine, then the processing time will be equal to the sum of their b_i , regardless of their order. Therefore it is only advantageous to send the jobs to the second machine in the same order as we sent them to the first machine.

Consider the order of the jobs, which coincides with their input order $1, 2, \dots, n$.

We denote by x_i the **idle time** of the second machine immediately before processing i . Our goal is to **minimize the total idle time**:

$$F(x) = \sum x_i \rightarrow \min$$

For the first job we have $x_1 = a_1$. For the second job, since it gets sent to the machine at the time $a_1 + a_2$, and the second machine gets free at $x_1 + b_1$, we have $x_2 = \max((a_1 + a_2) - (x_1 + b_1), 0)$. In general we get the equation:

$$x_k = \max\left(\sum_{i=1}^k a_i - \sum_{i=1}^{k-1} b_i - \sum_{i=1}^{k-1} x_i, 0\right)$$

We can now calculate the **total idle time** $F(x)$. It is claimed that it has the form

$$F(x) = \max_{k=1 \dots n} K_k,$$

where

$$K_k = \sum_{i=1}^k a_i - \sum_{i=1}^{k-1} b_i.$$

This can be easily verified using induction.

We now use the **permutation method**: we will exchange two neighboring jobs j and $j + 1$ and see how this will change the total idle time.

By the form of the expression of K_k , it is clear that only K_j and K_{j+1} change, we denote their new values with K'_j and K'_{j+1} .

If this change from of the jobs j and $j + 1$ increased the total idle time, it has to be the case that:

$$\max(K_j, K_{j+1}) \leq \max(K'_j, K'_{j+1})$$

(Switching two jobs might also have no impact at all. The above condition is only a sufficient one, but not a necessary one.)

After removing $\sum_{i=1}^{j+1} a_i - \sum_{i=1}^{j-1} b_i$ from both sides of the inequality, we get:

$$\max(-a_{j+1}, -b_j) \leq \max(-b_{j+1}, -a_j)$$

And after getting rid of the negative signs:

$$\min(a_j, b_{j+1}) \leq \min(b_j, a_{j+1})$$

Thus we obtained a **comparator**: by sorting the jobs on it, we obtain an optimal order of the jobs, in which no two jobs can be switched with an improvement of the final time.

However you can further **simplify** the sorting, if you look at the comparator from a different angle. The comparator can be interpreted in the following way: If we have the four times $(a_j, a_{j+1}, b_j, b_{j+1})$, and the minimum of them is a time corresponding to the first machine, then the corresponding job should be done first. If the minimum time is a time from the second machine, then it should go later. Thus we can sort the jobs by $\min(a_i, b_i)$, and if the processing time of the current job on the first machine is less than the processing time on the second machine, then this job must be done before all the remaining jobs, and otherwise after all remaining tasks.

One way or another, it turns out that by Johnson's rule we can solve the problem by sorting the jobs, and thus receive a time complexity of $O(n \log n)$.

Implementation

Here we implement the second variation of the described algorithm.

```
struct Job {
    int a, b, idx;

    bool operator<(Job o) const {
        return min(a, b) < min(o.a, o.b);
    }
};

vector<Job> johnsons_rule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());
    vector<Job> a, b;
    for (Job j : jobs) {
        if (j.a < j.b)
            a.push_back(j);
    }
}
```

```
        else
            b.push_back(j);
    }
    a.insert(a.end(), b.rbegin(), b.rend());
    return a;
}

pair<int, int> finish_times(vector<Job> const& jobs) {
    int t1 = 0, t2 = 0;
    for (Job j : jobs) {
        t1 += j.a;
        t2 = max(t2, t1) + j.b;
    }
    return make_pair(t1, t2);
}
```

All the information about each job is store in struct. The first function sorts all jobs and computes the optimal schedule. The second function computes the finish times of both machines given a schedule.

Contributors:

[Jakob Kogler](#) (94.12%) [Oleksandr Kulkov](#) (5.04%) [wiktorkuchta](#) (0.84%)

Translated

From: e-maxx.ru

Optimal schedule of jobs given their deadlines and durations

Suppose, we have a set of jobs, and we are aware of every job's deadline and its duration. The execution of a job cannot be interrupted prior to its ending. It is required to create such a schedule to accomplish the biggest number of jobs.

Solving

The algorithm of the solving is **greedy**. Let's sort all the jobs by their deadlines and look at them in descending order. Also, let's create a queue q , in which we'll gradually put the jobs and extract one with the least run-time (for instance, we can use set or priority_queue). Initially, q is empty.

Suppose, we're looking at the i -th job. First of all, let's put it into q . Let's consider the period of time between the deadline of i -th job and the deadline of $i - 1$ -th job. That is the segment of some length T . We will extract jobs from q (in their left duration ascending order) and execute them until the whole segment T is filled. Important: if at any moment of time the extracted job can only be partly executed until segment T is filled, then we execute this job partly just as far as possible, i.e., during the T -time, and we put the remaining part of a job back into q .

On the algorithm's completion we'll choose the optimal solution (or, at least, one of several solutions). The running time of algorithm is $O(n \log n)$.

Implementation

The following function takes a vector of jobs (consisting of a deadline, a duration, and the job's index) and computes a vector containing all indices of the used jobs in the optimal schedule. Notice that you still need to sort these jobs by their deadline, if you want to write down the plan explicitly.

```
struct Job {
    int deadline, duration, idx;

    bool operator<(Job o) const {
        return deadline < o.deadline;
    }
};

vector<int> compute_schedule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());

    set<pair<int,int>> s;
    vector<int> schedule;
    for (int i = jobs.size()-1; i >= 0; i--) {
        int t = jobs[i].deadline - (i ? jobs[i-1].deadline : 0);
        s.insert(make_pair(jobs[i].duration, jobs[i].idx));
        while (t && !s.empty()) {
            auto it = s.begin();
            if (it->first <= t) {
                t -= it->first;
                schedule.push_back(it->second);
            } else {
                s.insert(make_pair(it->first - t, it->second));
                t = 0;
            }
            s.erase(it);
        }
    }
    return schedule;
}
```

Contributors:

[Mariia Mykhailova](#) (45.45%) [Jakob Kogler](#) (43.64%) [Oleksandr Kulkov](#) (10.91%)

Translated

From: e-maxx.ru

Search the subarray with the maximum/minimum sum

Here, we consider the problem of finding a subarray with maximum sum, as well as some of its variations (including the algorithm for solving this problem online).

Problem statement

Given an array of numbers $a[1 \dots n]$. It is required to find a subarray $a[l \dots r]$ with the maximal sum:

$$\max_{1 \leq l \leq r \leq n} \sum_{i=l}^r a[i].$$

For example, if all integers in array $a[]$ were non-negative, then the answer would be the array itself. However, the solution is non-trivial when the array can contain both positive and negative numbers.

It is clear that the problem of finding the **minimum** subarray is essentially the same, you just need to change the signs of all numbers.

Algorithm 1

Here we consider an almost obvious algorithm. (Next, we'll look at another algorithm, which is a little harder to come up with, but its implementation is even shorter.)

Algorithm description

The algorithm is very simple.

We introduce for convenience the **notation**: $s[i] = \sum_{j=1}^i a[j]$. That is, the array $s[i]$ is an array of partial sums of array $a[]$. Also, set $s[0] = 0$.

Let us now iterate over the index $r = 1 \dots n$, and learn how to quickly find the optimal l for each current value r , at which the maximum sum is reached on the subarray $[l, r]$.

Formally, this means that for the current r we need to find an l (not exceeding r), so that the value of $s[r] - s[l - 1]$ is maximal. After a trivial transformation, we can see that we need to find in the array $s[]$ a minimum on the segment $[0, r - 1]$.

From here, we immediately obtain a solution: we simply store where the current minimum is in the array $s[]$. Using this minimum, we find the current optimal index l in $O(1)$, and when moving from the current index r to the next one, we simply update this minimum.

Obviously, this algorithm works in $O(n)$ and is asymptotically optimal.

Implementation

To implement it, we don't even need to explicitly store an array of partial sums $s[]$ — we will only need the current element from it.

The implementation is given in 0-indexed arrays, not in 1-numbering as described above.

We first give a solution that finds a simple numerical answer without finding the indices of the desired segment:

```
int ans = a[0], sum = 0, min_sum = 0;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    ans = max(ans, sum - min_sum);
    min_sum = min(min_sum, sum);
}
```

Now we give a full version of the solution, which additionally also finds the boundaries of the desired segment:

```
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, min_sum = 0, min_pos = -1;
```

```
for (int r = 0; r < n; ++r) {  
    sum += a[r];  
    int cur = sum - min_sum;  
    if (cur > ans) {  
        ans = cur;  
        ans_l = min_pos + 1;  
        ans_r = r;  
    }  
    if (sum < min_sum) {  
        min_sum = sum;  
        min_pos = r;  
    }  
}
```

Algorithm 2

Here we consider a different algorithm. It is a little more difficult to understand, but it is more elegant than the above, and its implementation is a little bit shorter. This algorithm was proposed by Jay Kadane in 1984.

Algorithm description

The algorithm itself is as follows. Let's go through the array and accumulate the current partial sum in some variable s . If at some point s is negative, we just assign $s = 0$. It is argued that the maximum all the values that the variable s is assigned to during the algorithm will be the answer to the problem.

Proof:

Consider the first index when the sum of s becomes negative. This means that starting with a zero partial sum, we eventually obtain a negative partial sum — so this whole prefix of the array, as well as any suffix, has a negative sum. Therefore, this subarray never contributes to the partial sum of any subarray of which it is a prefix, and can simply be dropped.

However, this is not enough to prove the algorithm. In the algorithm, we are actually limited in finding the answer only to such segments that begin immediately after the places when $s < 0$ happened.

But, in fact, consider an arbitrary segment $[l, r]$, and l is not in such a "critical" position (i.e. $l > p + 1$, where p is the last such position, in which $s < 0$). Since the last critical position is strictly earlier than in $l - 1$, it turns out that the sum of $a[p + 1 \dots l - 1]$ is non-negative. This means that by moving l to position $p + 1$, we will increase the answer or, in extreme cases, we will not change it.

One way or another, it turns out that when searching for an answer, you can limit yourself to only segments that begin immediately after the positions in which $s < 0$ appeared. This proves that the algorithm is correct.

Implementation

As in algorithm 1, we first gave a simplified implementation that looks for only a numerical answer without finding the boundaries of the desired segment:

```
int ans = a[0], sum = 0;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    ans = max(ans, sum);
    sum = max(sum, 0);
}
```

A complete solution, maintaining the indexes of the boundaries of the corresponding segment:

```
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, minus_pos = -1;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    if (sum > ans) {
        ans = sum;
        ans_l = minus_pos + 1;
        ans_r = r;
    }
    if (sum < 0) {
        sum = 0;
        minus_pos = r;
    }
}
```

Related tasks

Finding the maximum/minimum subarray with constraints

If the problem condition imposes additional restrictions on the required segment $[l, r]$ (for example, that the length $r - l + 1$ of the segment must be within the specified limits), then the described algorithm is likely to be easily generalized to these cases — anyway, the problem will still be to find the minimum in the array $s[]$ with the specified additional restrictions.

Two-dimensional case of the problem: search for maximum/minimum submatrix

The problem described in this article is naturally generalized to large dimensions. For example, in a two-dimensional case, it turns into a search for such a submatrix $[l_1 \dots r_1, l_2 \dots r_2]$ of a given matrix, which has the maximum sum of numbers in it.

Using the solution for the one-dimensional case, it is easy to obtain a solution in $O(n^3)$ for the two-dimensions case: we iterate over all possible values of l_1 and r_1 , and calculate the sums from l_1 to r_1 in each row of the matrix. Now we have the one-dimensional problem of finding the indices l_2 and r_2 in this array, which can already be solved in linear time.

Faster algorithms for solving this problem are known, but they are not much faster than $O(n^3)$, and are very complex (so complex that many of them are inferior to the trivial algorithm for all reasonable constraints by the hidden constant). Currently, the best known algorithm works in $O\left(n^3 \frac{\log^3 \log n}{\log^2 n}\right)$ time (T. Chan 2007 "More algorithms for all-pairs shortest paths in weighted graphs")

This algorithm by Chan, as well as many other results in this area, actually describe **fast matrix multiplication** (where matrix multiplication means modified multiplication: minimum is used instead of addition, and addition is used instead of multiplication). The problem of finding the submatrix with the largest sum can be reduced to the problem of finding the shortest paths between all pairs of vertices, and this problem, in turn, can be reduced to such a multiplication of matrices.

Search for a subarray with a maximum/minimum average

This problem lies in finding such a segment $a[l, r]$, such that the average value is maximal:

$$\max_{l \leq r} \frac{1}{r - l + 1} \sum_{i=l}^r a[i].$$

Of course, if no other conditions are imposed on the required segment $[l, r]$, then the solution will always be a segment of length 1 at the maximum element of the array. The problem only makes sense, if there are additional restrictions (for example, the length of the desired segment is bounded below).

In this case, we apply the **standard technique** when working with the problems of the average value: we will select the desired maximum average value by **binary search**.

To do this, we need to learn how to solve the following subproblem: given the number x , and we need to check whether there is a subarray of array $a[]$ (of course, satisfying all additional constraints of the problem), where the average value is greater than x .

To solve this subproblem, subtract x from each element of array $a[]$. Then our subproblem actually turns into this one: whether or not there are positive sum subarrays in this array. And we already know how to solve this problem.

Thus, we obtained the solution for the asymptotic $O(T(n) \log W)$, where W is the required accuracy, $T(n)$ is the time of solving the subtask for an array of length n (which may vary depending on the specific additional restrictions imposed).

Solving the online problem

The condition of the problem is as follows: given an array of n numbers, and a number L . There are queries of the form (l, r) , and in response to each query, it is required to find a subarray of the segment $[l, r]$ of length not less than L with the maximum possible arithmetic mean.

The algorithm for solving this problem is quite complex. KADR (Yaroslav Tverdokhlebov) described his algorithm on the [Russian forum](#).

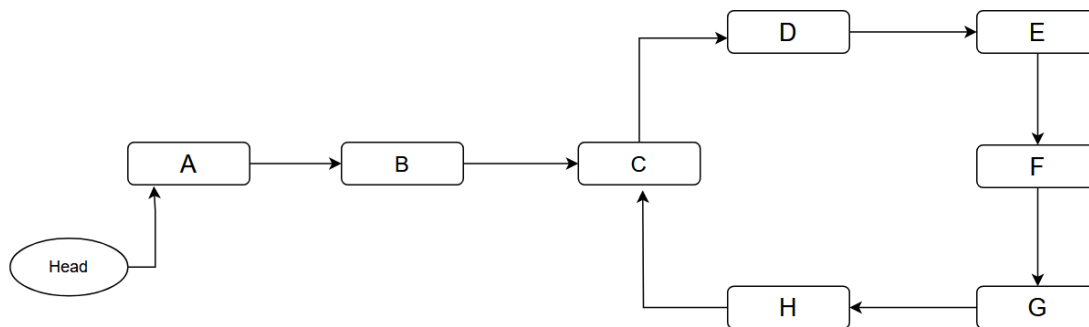
Contributors:

[Amandeep Singh](#) (79.88%) [Abhijeet Krishnan](#) (13.02%) [Oleksandr Kulkov](#) (3.55%)
[Anton Älgmyr](#) (2.37%) [Anish Aggarwal](#) (0.59%) [Tanmay Sinha](#) (0.59%)

Original

Floyd's Linked List Cycle Finding Algorithm

Given a linked list where the starting point of that linked list is denoted by **head**, and there may or may not be a cycle present. For instance:



Here we need to find out the point **C**, i.e the starting point of the cycle.

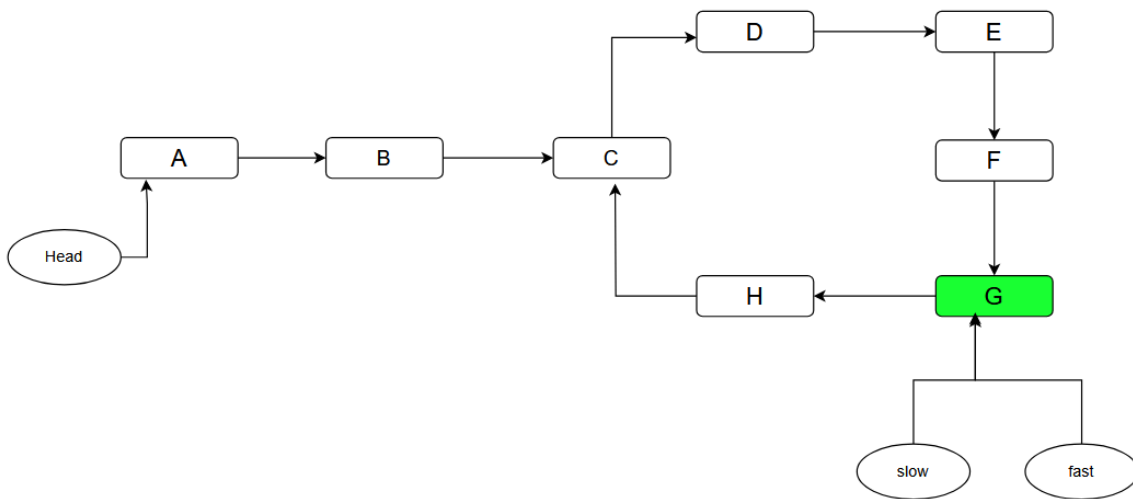
Proposed algorithm

The algorithm is called **Floyd's Cycle Algorithm** or **Tortoise And Hare algorithm**. In order to figure out the starting point of the cycle, we need to figure out if the cycle even exists or not. So, it involved two steps: 1. Figure out the presence of the cycle. 2. Find out the starting point of the cycle.

Step 1: Presence of the cycle

1. Take two pointers `slow` and `fast`.
2. Both of them will point to head of the linked list initially.
3. `slow` will move one step at a time.

4. `fast` will move two steps at a time. (twice as speed as `slow` pointer).
5. Check if at any point they point to the same node before any one(or both) reach `null`.
6. If they point to any same node at any point of their journey, it would indicate that the cycle indeed exists in the linked list.
7. If we get `null`, it would indicate that the linked list has no cycle.



Now, that we have figured out that there is a cycle present in the linked list, for the next step we need to find out the starting point of cycle, i.e., **C**.

Step 2: Starting point of the cycle

1. Reset the `slow` pointer to the **head** of the linked list.
2. Move both pointers one step at a time.
3. The point they will meet at will be the starting point of the cycle.

```
// Presence of cycle
public boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while(fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
    }
}
```

```

        if(slow==fast){
            return true;
        }
    }

    return false;
}

```

```

// Assuming there is a cycle present and slow and fast are point to
// their meeting point
slow = head;
while(slow!=fast){
    slow = slow.next;
    fast = fast.next;
}

return slow; // the starting point of the cycle.

```

Why does it work

Step 1: Presence of the cycle

Since the pointer `fast` is moving with twice as speed as `slow`, we can say that at any point of time, `fast` would have covered twice as much distance as `slow`. We can also deduce that the difference between the distance covered by both of these pointers is increasing by 1.

```

slow: 0 --> 1 --> 2 --> 3 --> 4 (distance covered)
fast: 0 --> 2 --> 4 --> 6 --> 8 (distance covered)
diff: 0 --> 1 --> 2 --> 3 --> 4 (difference between distance covered by
both pointers)

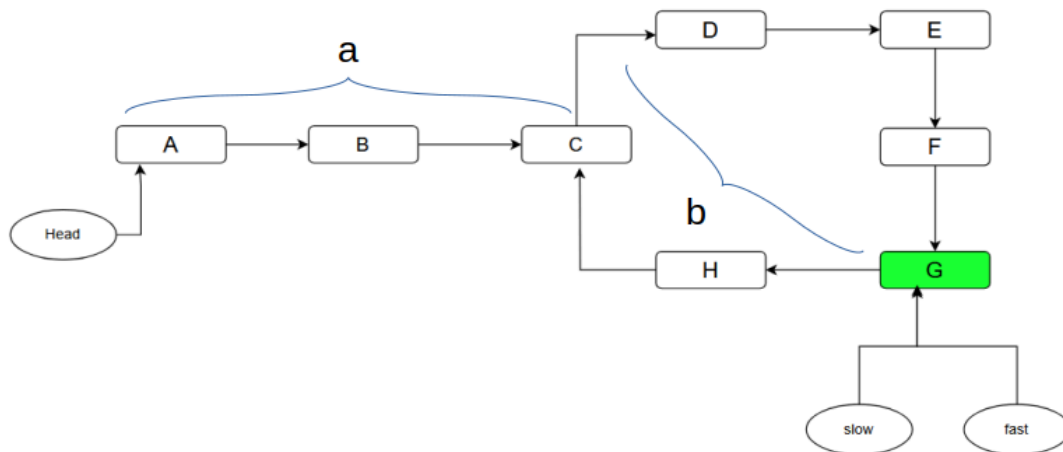
```

Let L denote the length of the cycle, and a represent the number of steps required for the slow pointer to reach the entry of cycle. There exists a positive integer k ($k > 0$) such that $k \cdot L \geq a$. When the slow pointer has moved $k \cdot L$ steps, and the fast pointer has covered $2 \cdot k \cdot L$ steps, both pointers find themselves within the cycle. At this point, there is a separation of $k \cdot L$ between them. Given that the cycle's length

remains L , this signifies that they meet at the same point within the cycle, resulting in their encounter.

Step 2: Starting point of the cycle

Lets try to calculate the distance covered by both of the pointers till they point they met within the cycle.



$$\text{slowDist} = a + xL + b, x \geq 0$$

$$\text{fastDist} = a + yL + b, y \geq 0$$

- slowDist is the total distance covered by slow pointer.
- fastDist is the total distance covered by fast pointer.
- a is the number of steps both pointers need to take to enter the cycle.
- b is the distance between **C** and **G**, i.e., distance between the starting point of cycle and meeting point of both pointers.
- x is the number of times the slow pointer has looped inside the cycle, starting from and ending at **C**.
- y is the number of times the fast pointer has looped inside the cycle, starting from and ending at **C**.

$$\text{fastDist} = 2 \cdot (\text{slowDist})$$

$$a + yL + b = 2(a + xL + b)$$

Resolving the formula we get:

$$a = (y - 2x)L - b$$

where $y - 2x$ is an integer

This basically means that a steps is same as doing some number of full loops in cycle and go b steps backwards. Since the fast pointer already is b steps ahead of the entry of cycle, if fast pointer moves another a steps it will end up at the entry of the cycle. And since we let the slow pointer start at the start of the linked list, after a steps it will also end up at the cycle entry. So, if they both move a step they both will meet the entry of cycle.

Problems:

- [Linked List Cycle \(EASY\)](#)
- [Happy Number \(Easy\)](#)
- [Find the Duplicate Number \(Medium\)](#)

Contributors:

[Sourav](#) (88.43%) [Michael Hayter](#) (7.44%) [Kanwae Kan](#) (2.48%) [Isaac Bode](#) (0.83%)
[ericmiranda7](#) (0.83%)

Translated

From: e-maxx.ru

Josephus Problem

Statement

We are given the natural numbers n and k . All natural numbers from 1 to n are written in a circle. First, count the k -th number starting from the first one and delete it. Then k numbers are counted starting from the next one and the k -th one is removed again, and so on. The process stops when one number remains. It is required to find the last number.

This task was set by **Flavius Josephus** in the 1st century (though in a somewhat narrower formulation: for $k = 2$).

This problem can be solved by modeling the procedure. Brute force modeling will work $O(n^2)$. Using a [Segment Tree](#), we can improve it to $O(n \log n)$. We want something better though.

Modeling a $O(n)$ solution

We will try to find a pattern expressing the answer for the problem $J_{n,k}$ through the solution of the previous problems.

Using brute force modeling we can construct a table of values, for example, the following:

n \ k	1	2	3	4	5	6	7	8	9	10
1	1	1	1	1	1	1	1	1	1	1
2	2	1	2	1	2	1	2	1	2	1
3	3	3	2	2	1	1	3	3	2	2
4	4	1	1	2	2	3	2	3	3	4
5	5	3	4	1	2	4	4	1	2	4
6	6	5	1	5	1	4	5	3	5	2
7	7	7	4	2	6	3	5	4	7	5
8	8	1	7	6	3	1	4	4	8	7
9	9	3	1	1	8	7	2	3	8	8
10	10	5	4	5	3	3	9	1	7	8

And here we can clearly see the following **pattern**:

$$J_{n,k} = ((J_{n-1,k} + k - 1) \bmod n) + 1$$

$$J_{1,k} = 1$$

Here, 1-indexing makes for a somewhat messy formula; if you instead number the positions from 0, you get a very elegant formula:

$$J_{n,k} = (J_{n-1,k} + k) \bmod n$$

So, we found a solution to the problem of Josephus, working in $O(n)$ operations.

Implementation

Simple **recursive implementation** (in 1-indexing)

```
int josephus(int n, int k) {
    return n > 1 ? (josephus(n-1, k) + k - 1) % n + 1 : 1;
}
```

Non-recursive form :

```
int josephus(int n, int k) {
    int res = 0;
    for (int i = 1; i <= n; ++i)
        res = (res + k) % i;
    return res + 1;
}
```

This formula can also be found analytically. Again here we assume 0-indexing. After we delete the first number, we have $n - 1$ numbers left. When we repeat the procedure, we will start with the number that had originally the index $k \bmod n$. $J_{n-1,k}$ would be the answer for the remaining circle, if we start counting at 0, but because we actually start with k we have $J_{n,k} = (J_{n-1,k} + k) \bmod n$.

Modeling a $O(k \log n)$ solution

For relatively small k we can come up with a better solution than the above recursive solution in $O(n)$. If k is a lot smaller than n , then we can delete multiple numbers ($\lfloor \frac{n}{k} \rfloor$) in one run without looping over. Afterwards we have $n - \lfloor \frac{n}{k} \rfloor$ numbers left, and we start with the $(\lfloor \frac{n}{k} \rfloor \cdot k)$ -th number. So we have to shift by that many. We can notice that $\lfloor \frac{n}{k} \rfloor \cdot k$ is simply $-n \bmod k$. And because we removed every k -th number, we have to add the number of numbers that we removed before the result index. Which we can compute by dividing the result index by $k - 1$.

Also, we need to handle the case when n becomes less than k . In this case, the above optimization would cause an infinite loop.

Implementation (for convenience in 0-indexing):

```
int josephus(int n, int k) {
    if (n == 1)
        return 0;
    if (k == 1)
        return n-1;
    if (k > n)
        return (josephus(n-1, k) + k) % n;
    int cnt = n / k;
    int res = josephus(n - cnt, k);
```

```

    res -= n % k;
    if (res < 0)
        res += n;
    else
        res += res / (k - 1);
    return res;
}

```

Let us estimate the **complexity** of this algorithm. Immediately note that the case $n < k$ is analyzed by the old solution, which will work in this case for $O(k)$. Now consider the algorithm itself. In fact, after every iteration, instead of n numbers, we are left with $n(1 - \frac{1}{k})$ numbers, so the total number of iterations x of the algorithm can be found roughly from the following equation:

$$n(1 - \frac{1}{k})^x = 1,$$

on taking logarithm on both sides, we obtain:

$$\ln n + x \ln(1 - \frac{1}{k}) = 0,$$

$$x = -\frac{\ln n}{\ln(1 - \frac{1}{k})},$$

using the decomposition of the logarithm into Taylor series, we obtain an approximate estimate:

$$x \approx k \ln n$$

Thus, the complexity of the algorithm is actually $O(k \log n)$.

Analytical solution for $k = 2$

In this particular case (in which this task was set by Josephus Flavius) the problem is solved much easier.

In the case of even n we get that all even numbers will be crossed out, and then there will be a problem remaining for $\frac{n}{2}$, then the answer for n will be obtained from the answer for $\frac{n}{2}$ by multiplying by two and subtracting one (by shifting positions):

$$J_{2n,2} = 2J_{n,2} - 1$$

Similarly, in the case of an odd n , all even numbers will be crossed out, then the first number, and the problem for $\frac{n-1}{2}$ will remain, and taking into account the shift of positions, we obtain the second formula:

$$J_{2n+1,2} = 2J_{n,2} + 1$$

We can use this recurrent dependency directly in our implementation. This pattern can be translated into another form: $J_{n,2}$ represents a sequence of all odd numbers, "restarting" from one whenever n turns out to be a power of two. This can be written as a single formula:

$$J_{n,2} = 1 + 2(n - 2^{\lfloor \log_2 n \rfloor})$$

Analytical solution for $k > 2$

Despite the simple form of the problem and a large number of articles on this and related problems, a simple analytical representation of the solution of Josephus' problem has not yet been found. For small k , some formulas are derived, but apparently they are all difficult to apply in practice (for example, see Halbeisen, Hungerbuhler "The Josephus Problem" and Odlyzko, Wilf "Functional iteration and the Josephus problem").

Contributors:

[Amandeep Singh](#) (63.76%) [Jakob Kogler](#) (16.78%) [Abhijeet Krishnan](#) (12.75%)
[Oleksandr Kulkov](#) (4.7%) [Anton Älgmyr](#) (1.34%) [Dikson Santos](#) (0.67%)

Translated

From: e-maxx.ru

15 Puzzle Game: Existence Of The Solution

This game is played on a 4×4 board. On this board there are 15 playing tiles numbered from 1 to 15. One cell is left empty (denoted by 0). You need to get the board to the position presented below by repeatedly moving one of the tiles to the free space:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	0

The game "15 Puzzle" was created by Noyes Chapman in 1880.

Existence Of The Solution

Let's consider this problem: given a position on the board, determine whether a sequence of moves which leads to a solution exists.

Suppose we have some position on the board:

a_1	a_2	a_3	a_4
a_5	a_6	a_7	a_8
a_9	a_{10}	a_{11}	a_{12}
a_{13}	a_{14}	a_{15}	a_{16}

where one of the elements equals zero and indicates an empty cell $a_z = 0$

Let's consider the permutation:

$$a_1 a_2 \dots a_{z-1} a_{z+1} \dots a_{15} a_{16}$$

i.e. the permutation of numbers corresponding to the position on the board without a zero element

Let N be the number of inversions in this permutation (i.e. the number of such elements a_i and a_j that $i < j$, but $a_i > a_j$).

Suppose K is an index of a row where the empty element is located (i.e. using our convention, $K = (z - 1) \div 4 + 1$).

Then, **the solution exists iff $N + K$ is even.**

Implementation

The algorithm above can be illustrated with the following program code:

```
int a[16];
for (int i=0; i<16; ++i)
    cin >> a[i];

int inv = 0;
for (int i=0; i<16; ++i)
    if (a[i])
        for (int j=0; j<i; ++j)
            if (a[j] > a[i])
                ++inv;
for (int i=0; i<16; ++i)
    if (a[i] == 0)
        inv += 1 + i / 4;

puts ((inv & 1) ? "No Solution" : "Solution Exists");
```

Proof

In 1879 Johnson proved that if $N + K$ is odd, then the solution doesn't exist, and in the same year Story proved that all positions when $N + K$ is even have a solution.

However, all these proofs were quite complex.

In 1999 Archer proposed a much simpler proof (you can download his article [here](#)).

Practice Problems

- [Hackerrank - N-puzzle](#)

Contributors:

[ellen-interpret](#) (68.12%) [Mariia Mykhailova](#) (14.49%) [Oleksandr Kulkov](#) (8.7%)
[Abhijeet Krishnan](#) (5.8%) [Jakob Kogler](#) (2.9%)

Translated

From: e-maxx.ru

The Stern-Brocot tree and Farey sequences

Stern-Brocot tree

The Stern-Brocot tree is an elegant construction to represent the set of all positive fractions. It was independently discovered by German mathematician Moritz Stern in 1858 and by French watchmaker Achille Brocot in 1861. However, some sources attribute the discovery to ancient Greek mathematician Eratosthenes.

The construction starts at the zeroth iteration with the two fractions

$$\frac{0}{1}, \frac{1}{0}$$

where it should be noted that the second quantity is not strictly a fraction, but it can be interpreted as an irreducible fraction representing infinity.

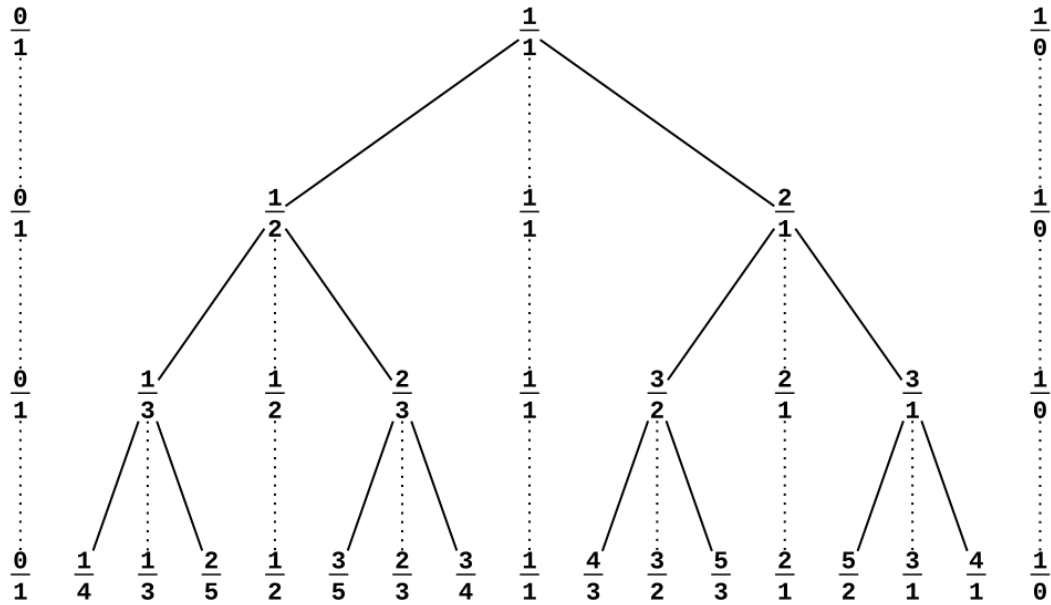
At every subsequent iteration, consider all adjacent fractions $\frac{a}{b}$ and $\frac{c}{d}$ and insert their **mediant** $\frac{a+c}{b+d}$ between them.

The first few iterations look like this:

$$\begin{array}{c} \frac{0}{1}, \frac{1}{1}, \frac{1}{0} \\ \frac{0}{1}, \frac{1}{2}, \frac{1}{1}, \frac{2}{1}, \frac{1}{0} \\ \frac{0}{1}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{1}{1}, \frac{3}{2}, \frac{2}{1}, \frac{3}{1}, \frac{1}{0} \end{array}$$

Continuing this process to infinity this covers *all* positive fractions. Additionally, all fractions will be *unique* and *irreducible*. Finally, the fractions will also appear in ascending order.

Before proving these properties, let us actually show a visualization of the Stern-Brocot tree, rather than the list representation. Every fraction in the tree has two children. Each child is the mediant of the closest ancestor on the left and closest ancestor to the right.



Proofs

Ordering. Proving ordering is simple. We note that the mediant of two fractions is always in-between the fractions

$$\frac{a}{b} \leq \frac{a+c}{b+d} \leq \frac{c}{d}$$

given that

$$\frac{a}{b} \leq \frac{c}{d}.$$

The two inequalities can be easily shown by rewriting the fractions with common denominators.

As the ordering is ascending in the zeroth iteration, it will be maintained at every subsequent iteration.

Irreducibility. To prove this we will show that for any two adjacent fractions $\frac{a}{b}$ and $\frac{c}{d}$ we have that

$$bc - ad = 1.$$

Recall that a Diophantine equation with two variables $ax + by = c$ has a solution iff c is a multiple of $\gcd(a, b)$. In our case this implies that $\gcd(a, b) = \gcd(c, d) = 1$, which is what we want to show.

Clearly at the zeroth iteration $bc - ad = 1$. What remains to be shown is that mediants retain this property.

Assume our two adjacent fractions uphold $bc - ad = 1$, after the mediant is added to the list

$$\frac{a}{b}, \frac{a+c}{b+d}, \frac{c}{d}$$

the new expressions become

$$b(a+c) - a(b+d) = 1$$

$$c(b+d) - d(a+c) = 1$$

which, using that $bc - ad = 1$, can be easily shown to be true.

From this we see that the property is always maintained and thus all fractions are irreducible.

The presence of all fractions. This proof is closely related to locating a fraction in the Stern-Brocot tree. From the ordering property we have that left subtree of a fraction

contains only fractions smaller than the parent fraction, and the right subtree contains only fractions larger than the parent fraction. This means we can search for a fraction by traversing the tree from the root, going left if the target is smaller than the fraction and going right if the target is larger.

Pick an arbitrary positive target fraction $\frac{x}{y}$. It is obviously between $\frac{0}{1}$ and $\frac{1}{0}$, so the only way for the fraction to not be in the tree is if it takes an infinite number of steps to get to it.

If that is the case we would at all iterations have

$$\frac{a}{b} < \frac{x}{y} < \frac{c}{d}$$

which (using the fact that an integer $z > 0 \iff z \geq 1$) can be rewritten as

$$\begin{aligned} bx - ay &\geq 1 \\ cy - dx &\geq 1. \end{aligned}$$

Now multiply the first inequality by $c + d$ and the second with $a + b$ and add them to get

$$(c + d)(bx - ay) + (a + b)(cy - dx) \geq a + b + c + d.$$

Expanding this and using the previously shown property $bc - ad = 1$ we get that

$$x + y \geq a + b + c + d.$$

And given that at every iteration at least one of a, b, c, d will increase, the fraction searching process will contain no more than $x + y$ iterations. This contradicts the assumption that the path to $\frac{x}{y}$ was infinite and hence $\frac{x}{y}$ must be part of the tree.

Tree Building Algorithm

To build any subtree of the Stern-Brocot tree, it suffices to know the left and right ancestor. On the first level, the left and right ancestors are $\frac{0}{1}$ and $\frac{1}{0}$ respectively. Using these, we calculate the mediant and proceed one level deeper, with the mediant replacing the right ancestor in the left subtree, and vice versa.

This pseudocode tries to build the entire infinite tree:

```
void build(int a = 0, int b = 1, int c = 1, int d = 0, int level = 1) {
    int x = a + c, y = b + d;

    ... output the current fraction x/y at the current level in the tree

    build(a, b, x, y, level + 1);
    build(x, y, c, d, level + 1);
}
```

Fraction Search Algorithm

The search algorithm was already described in the proof that all fractions appear in the tree, but we will repeat it here. The algorithm is a binary search algorithm. Initially we stand at the root of the tree and we compare our target with the current fraction. If they are the same we are done and stop the process. If our target is smaller we move to the left child, otherwise we move to the right child.

Naive search

Here is an implementation that returns the path to a given fraction $\frac{p}{q}$ as a sequence of 'L' and 'R' characters, meaning traversal to the left and right child respectively. This sequence of characters uniquely defines all positive fractions and is called the Stern-Brocot number system.

```
string find(int p, int q) {
    int pL = 0, qL = 1;
    int pR = 1, qR = 0;
    int pM = 1, qM = 1;
    string res;
    while(pM != p || qM != q) {
        if(p * qM < pM * q) {
            res += 'L';
            tie(pR, qR) = {pM, qM};
        }
    }
```

```

    } else {
        res += 'R';
        tie(pL, qL) = {pM, qM};
    }
    tie(pM, qM) = pair{pL + pR, qL + qR};
}
return res;
}

```

Irrational numbers in the Stern-Brocot number system corresponds to infinite sequences of characters. Along the endless path towards the irrational number the algorithm will find reduced fractions with gradually increasing denominators that provides increasingly better approximations of the irrational number. So by taking a prefix of the infinite sequence approximations with any desired precision can be achieved. This application is important in watch-making, which explains why the tree was discovered in that domain.

Note that for a fraction $\frac{p}{q}$, the length of the resulting sequence could be as large as $O(p + q)$, for example when the fraction is of form $\frac{p}{1}$. This means that the algorithm above **should not be used, unless this is an acceptable complexity!**

Logarithmic search

Fortunately, it is possible to enhance the algorithm above to guarantee $O(\log(p + q))$ complexity. For this we should note that if the current boundary fractions are $\frac{p_L}{q_L}$ and $\frac{p_R}{q_R}$, then by doing a steps to the right we move to the fraction $\frac{p_L + a p_R}{q_L + a q_R}$, and by doing a steps to the left, we move to the fraction $\frac{a p_L + p_R}{a q_L + q_R}$.

Therefore, instead of doing steps of **L** or **R** one by one, we can do k steps in the same direction at once, after which we would switch to going into other direction, and so on. In this way, we can find the path to the fraction $\frac{p}{q}$ as its run-length encoding.

As the directions alternate this way, we will always know which one to take. So, for convenience we may represent a path to a fraction $\frac{p}{q}$ as a sequence of fractions

$$\frac{p_0}{q_0}, \frac{p_1}{q_1}, \frac{p_2}{q_2}, \dots, \frac{p_n}{q_n}, \frac{p_{n+1}}{q_{n+1}} = \frac{p}{q}$$

such that $\frac{p_{k-1}}{q_{k-1}}$ and $\frac{p_k}{q_k}$ are the boundaries of the search interval on the k -th step, starting with $\frac{p_0}{q_0} = \frac{0}{1}$ and $\frac{p_1}{q_1} = \frac{1}{0}$. Then, after the k -th step we move to a fraction

$$\frac{p_{k+1}}{q_{k+1}} = \frac{p_{k-1} + a_k p_k}{q_{k-1} + a_k q_k},$$

where a_k is a positive integer number. If you're familiar with [continued fractions](#), you would recognize that the sequence $\frac{p_i}{q_i}$ is the sequence of the convergent fractions of $\frac{p}{q}$ and the sequence $[a_1; a_2, \dots, a_n, 1]$ represents the continued fraction of $\frac{p}{q}$.

This allows to find the run-length encoding of the path to $\frac{p}{q}$ in the manner which follows the algorithm for computing continued fraction representation of the fraction $\frac{p}{q}$:

```
auto find(int p, int q) {
    bool right = true;
    vector<pair<int, char>> res;
    while(q) {
        res.emplace_back(p / q, right ? 'R' : 'L');
        tie(p, q) = pair{q, p % q};
        right ^= 1;
    }
    res.back().first--;
    return res;
}
```

However, this approach only works if we already know $\frac{p}{q}$ and want to find its place in the Stern-Brocot tree.

On practice, it is often the case that $\frac{p}{q}$ is not known in advance, but we are able to check for specific $\frac{x}{y}$ whether $\frac{x}{y} < \frac{p}{q}$.

Knowing this, we can emulate the search on Stern-Brocot tree by maintaining the current boundaries $\frac{p_{k-1}}{q_{k-1}}$ and $\frac{p_k}{q_k}$, and finding each a_k via binary search. The algorithm then is a bit more technical and potentially have a complexity of $O(\log^2(x + y))$, unless the problem formulation allows you to find a_k faster (for example, using `floor` of some known expression).

Farey Sequence

The Farey sequence of order n is the sorted sequence of fractions between 0 and 1 whose denominators do not exceed n .

The sequences are named after English geologist John Farey, who in 1816 conjectured that any fraction in a Farey sequence is the mediant of its neighbors. This was proven some time later by Cauchy, but independent of both of them, the mathematician Haros had come to almost the same conclusion in 1802.

The Farey sequences have many interesting properties on their own, but the connection to the Stern-Brocot tree is the most obvious. In fact, the Farey sequences can be obtained by trimming branches from the tree.

From the algorithm for building the Stern-Brocot tree, we get an algorithm for the Farey sequences. Start with the list of fractions $\frac{0}{1}, \frac{1}{0}$. At every subsequent iteration, insert the mediant only if the denominator does not exceed n . At some point the list will stop changing and the desired Farey sequence will have been found.

Length of a Farey Sequence

A Farey sequence of order n contains all elements of the Farey sequence of order $n - 1$ as well as all irreducible fractions with denominator n , but the latter is just the totient $\varphi(n)$. So the length L_n of the Farey sequence of order n is

$$L_n = L_{n-1} + \varphi(n)$$

or equivalently, by unraveling the recursion we get

$$L_n = 1 + \sum_{k=1}^n \varphi(k).$$

Contributors:

[Anton Älgmyr](#) (50.86%) [Oleksandr Kulkov](#) (29.74%) [Abhijeet Krishnan](#) (16.38%)
[Jakob Kogler](#) (1.72%) [Michael Hayter](#) (1.29%)